UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos e Programação (APE)**

Aula: 06 - Bibliotecas para Tratamento de Dados

IA PARA ENGENHEIROS
UFRGS

Professor: **Raphael Martins Brum**

2026

Biblioteca NumPy

- *Numerical Python extensions*
- Biblioteca para computação numérica e científica
- Implementa o **ndarray** *(n-dimensional arrays)*, alternativa às estruturas *built-in*
- Permite realizar **operações vetoriais** sobre os ndarrays
- Implementa uma série de funções matemáticas que trabalham sobre essa estrutura, em C/Fortran
- Semelhança com a sintaxe e modo de funcionar do MATLAB

Criada por Travis Oliphant em 2005

Base de pandas e outras

Primeiro exemplo

```python
import numpy as np

lista = [1, 2, 3, 4, 5]
arr   = np.array(lista)

print(arr)
print(arr * 2)
print(arr ** 2)
```

Saídas

```
[ 1 2 3 4 5 ]
[ 2 4 6 8 10 ]
[ 1 4 9 16 25 ]
```

Comparação de Desempenho

```
import numpy as np
import time
n = 10_000_000
lista = list(range(n))
arr   = np.arange(n)
t0 = time.perf_counter();
sum(lista);
t_list = time.perf_counter() - t0
t0 = time.perf_counter();
arr.sum();
t_np   = time.perf_counter() - t0
print(f"Lista : {t_list*1000:.1f} ms")
print(f"NumPy : {t_np*1000:.1f} ms")
```

Atributos essenciais do ndarray

```
a = np.array([[1, 2, 3],
              [4, 5, 6]], dtype=np.float32)

a.shape    # (2, 3)  – dimensões
a.ndim     # 2       – número de eixos
a.size     # 6       – total de elementos
a.dtype    # float32 – tipo dos dados
a.itemsize # 4       – bytes por elemento
a.nbytes   # 24      – total em bytes
a.strides  # (12, 4) – bytes p/ avançar
a.flags    # C_CONTIGUOUS, WRITEABLE…
```

Tipos de dados definidos pela biblioteca

```python
# Inteiros
np.int8, np.int16, np.int32, np.int64
np.uint8, np.uint16  # sem sinal

# Ponto flutuante
np.float16, np.float32, np.float64

# Complexos e booleano
np.complex64, np.complex128
np.bool_
```

```python
# Conversão
a.astype(np.int32)
np.array([1.5, 2.7], dtype="int")
# [1 2]  - trunca, não arredonda
```

Criando arrays a partir de estruturas básicas

```
a = np.array([10, 20, 30])

m = np.array([[1, 2, 3],
              [4, 5, 6]])

print(m.shape)
print(m.dtype)
print(m.ndim)
```

Saídas

```
(2, 3)
int64
2
```

Criando arrays a partir de “geradores”

```
# Zeros e uns
np.zeros((3, 3))
np.ones((2, 4))
#np.full((2,2), 7)
np.eye(3)
# Sequências
np.arange(0, 10, 2)
np.linspace(0, 1, 5)
#np.logspace(0,3,4)
#np.geomspace(1,1000,4)
# Aleatórios
np.random.rand(3, 3)
np.random.randn(100)
```

Saídas

```
# matriz 3×3 de 0s
# matriz 2×4 de 1s
# identidade 3×3

# [0 2 4 6 8]
# [0. .25 .5 .75 1.]
# uniforme [0,1)
# distribuição normal
```

Indexação e Fatiamento (*slicing*)

```
m = np.array([[1,  2,  3,  4],
              [5,  6,  7,  8],
              [9, 10, 11, 12]])
m[0, 0]   # 1
m[1, 2]   # 7
m[-1, -1] # 12

m[0, :]    # 1ª linha
m[:, 1]    # 2ª coluna
m[0:2, 1:3] # submatriz

m[m > 6]
m[m % 2 == 0]
```

Saídas

```
# 1, 7, 12
# 1ª col, 2ª col, submat

# array([7,8,9,10,11,12])
# pares: [2,4,6,8,10,12]
```

Fancy indexing, boolean indexing, where

```python
a = np.arange(10) * 10

idx = [1, 3, 7]
a[idx]

m = np.arange(16).reshape(4,4)
linhas = [0, 2]
colunas= [1, 3]

m[linhas, colunas]
m[np.ix_(linhas, colunas)]
```

```python
v = np.array([4, -2, 7, -1, 9, 0])

# Filtrar
v[v > 0]              # [4 7 9]
v[(v > 0) & (v < 8)] # [4 7]

# np.where - ternário vetorial
np.where(v > 0, v, 0)
# [4 0 7 0 9 0]  - relu!

# Localizar índices verdadeiros
np.where(v < 0)    # (array([1,3]),)
np.nonzero(v)       # equivalente
np.argwhere(v < 0) # [[1],[3]]
```

Operações matemáticas - elemento a elemento

```python
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b
a * b
a / b
a ** 2
np.sqrt(a)
np.exp(a)
```

Saídas

```
# [5 7 9]
# [4 10 18]
# [0.25 0.4 0.5]
# [1 4 9]
# [1.  1.41 1.73]
# [2.72 7.39 20.09]
```

Operações matemáticas - agregação

```
v = np.array([3, 1, 4, 1, 5, 9,
2, 6])

v.sum()
v.mean()
v.std()
v.min()
v.max()
v.argmax()
np.median(v)
```

Saídas

```
# 31
# 3.875
# 2.588
# 1
# 9
# 5 (5º)
# 3.5
```

Broadcasting - operação em matrizes de ordem diferente

```python
# Escalar x array

a = np.arange(6).reshape(2, 3)
a + 10

# Linha x Coluna

linha = np.array([[0, 1, 2]])
col   = np.array([[0],[10],[20]])
linha + col
```

Saídas

```
# [[10 11 12]
#  [13 14 15]]

# [[ 0  1  2]
#  [10 11 12]
#  [20 21 22]]
```

Ordenação, busca e contagem

```
v = np.array([3, 1, 4, 1, 5, 9, 2])

np.sort(v)
v.sort()
np.argsort(v)

m = np.array([[3,1],[2,4],[1,2]])
m[m[:,0].argsort()]

np.partition(v, 3)
np.argpartition(v, -3)[-3:]
```

```
a = np.array([10, 20, 30, 20, 10])

np.unique(a)
np.unique(a, return_counts=True)

np.searchsorted(np.sort(a), 15)

np.count_nonzero(a > 15)
np.any(a > 25)
np.all(a > 5)
np.in1d([10,99], a)
```

Manipulação do formato dos arrays (1)

```python
a = np.arange(12)


a.reshape(3, 4)
a.reshape(2, -1) # -1 = inferido
(2,6)
a.reshape(2, 3, 2) # 3D

a.flatten()
a.ravel()
```

Saídas

```
# [ 0  1  2  3  4  5  6  7  8
9 10 11]

# matriz 3x4
# matriz 3x6
# 2x3x2

# flatten – 1D
# view 1D (sem copia)
```

Manipulação do formato dos arrays (2)

```
x = np.array([[1, 2], [3, 4]])
y = np.array([[5, 6], [7, 8]])

np.vstack([x, y])
np.hstack([x, y])

partes = np.split(x, 2, axis=0)


a.T
```

Saídas

```
# empilha linhas
# empilha colunas

# [array([[1,2]]),
array([[3,4]])]

# transposta
```

Exemplos de Álgebra Linear

```python
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A @ B

np.linalg.norm(A)
np.linalg.det(A)
np.linalg.matrix_rank(A)
```

Saídas

```
# produto matricial
# [[19 22] [43 50]]

# demais: rodar
```

Exemplos de Estatística

```
rng = np.random.default_rng(seed=42)
dados = rng.normal(loc=170, scale=10,
size=(50, 3))

dados.mean(axis=0)
dados.std(axis=0)
dados.min(axis=1)

np.percentile(dados, [25, 50, 75])
np.corrcoef(dados.T)
```

```
rng = np.random.default_rng(0)
# Normal padrão
rng.standard_normal(1000)

# Binomial
rng.binomial(n=10, p=0.5, size=500)

# Inteiros aleatórios
rng.integers(1, 7, size=20)  # dado

# Amostragem
rng.choice([1,2,3,4], size=10,
           replace=True)
```

Más práticas x boas práticas

```python
# RUIM — loop Python sobre array
resultado = []
for x in arr:
    resultado.append(np.sin(x) * 2)
# RUIM — append dentro de loop
saida = np.array([])
for v in arr:
    saida = np.append(saida, v**2)
# cópia a cada iteração!

# RUIM — float64 desnecessário
a = np.zeros(1000, dtype=np.float64)
# use float32 se precisão não importa
```

```python
# BOM — operação vetorizada
resultado = np.sin(arr) * 2

# BOM — pré-alocar
saida = np.empty(len(arr))
saida = arr**2

# BOM — np.vectorize
def minha_func(x):
    return x**2 if x > 0 else 0
vf = np.vectorize(minha_func)
np.select([arr<0, arr==0, arr>0],
          [-1, 0, 1])
```

Exercício (15 min + 15 min intervalo)

Vamos utilizar o código abaixo para gerar dados sintéticos sobre o desempenho de 100 alunos em quatro disciplinas. Utilize os métodos da NumPy para calcular a média e o desvio padrão por disciplina.

Em seguida, adotando a convenção de conceitos A (nota entre 90 e 100), B (nota entre 75 e 89), C (nota entre 60 e 74) ou D (nota entre zero e 59), determine o número de conceitos A, B, C e D em cada disciplina. **Não use loops nativos do Python para as operações sobre os arrays!**

```
rng = np.random.default_rng(seed=42)
notas = rng.normal(loc=65, scale=15, size=(100, 4))
notas = np.clip(notas, 0, 100)   # garante [0, 100]
disciplinas = ["Matemática", "Física", "Química", "Biologia"]
```

Mais sobre NumPy

- FFT, Funções Janela
- Derivadas numéricas
- Formato NPZ (formato binário de armazenamento dos ndarrays)
- e muito mais…

- Documentação https://numpy.org/doc/stable/

Biblioteca Pandas

- *Panel Data*
- Biblioteca para manipulação de séries de dados (dados tabulados, tipicamente)
- Implementa as estruturas *series* e *DataFrame*
- Trabalha nativamente com CSV, Excel, SQL, JSON
- Filtra, agrupa e faz estatísticas com sintaxe concisa
- Integra-se com *NumPy*, *matplotlib* e outras bibliotecas relevantes para tratamento de dados

Criada por Wes McKinney em 2008

Estrutura de Dados *Series*

```python
# Criando Series de várias formas
s1 = pd.Series([10, 20, 30, 40])
print(s1)

# Com índice customizado
notas = pd.Series(
    [9.5, 7.0, 8.2],
    index=["Ana", "Bruno", "Carla"]
)

# Acessando valores
notas["Ana"]      # → 9.5
notas.iloc[0]   # → 9.5 (posição)
```

Saídas

```
# 0     10
# 1     20
# 2     30
# 3     40
```

Estrutura de Dados *Series* - operações vetorizadas

```python
# Multiplicação
print(notas * 10)

# Estatísticas básicas
notas.mean()
notas.max()
notas.std()

# Filtro booleano
notas[notas >= 8.0]
```

Saídas

```
# Ana       95.0
# Bruno     70.0
# Carla     82.0

# 8.23
# 9.5
# 1.28

# Ana       9.5
# Carla     8.2
```

Estrutura de Dados *DataFrame*

```python
# Criando a partir de dicionário
dados = {
    "nome":  ["Ana", "Bruno",
"Carla"],
    "nota":  [9.5, 7.0, 8.2],
    "turma": ["A", "B", "A"]
}
df = pd.DataFrame(dados)
print(df)
```

Saídas

```
   nome  nota turma
0   Ana   9.5     A
1 Bruno   7.0     B
2 Carla   8.2     A
```

Coluna do DataFrame = Series

DataFrame = dict de Series

Propriedades do *DataFrame*

```python
# Propriedades essenciais
df.shape
df.columns
df.dtypes
df.index

# Acessar colunas
df["nota"]
df[["nome","nota"]]

# Nova coluna
df["aprovado"] = df["nota"] >= 6.0
```

Saídas

```
# (3, 3) → linhas, colunas
# Index(['nome','nota','turma'])
# tipos de cada coluna
# RangeIndex(0, 3)

(...)
```

Construindo DataFrames com Arquivos

```python
# CSV
df = pd.read_csv("vendas.csv")

# Algumas opções
df = pd.read_csv(
    "vendas.csv",
    sep=";",
    encoding="utf-8",
    parse_dates=["data"],
    index_col="id"
)
```

```python
# Excel
df = pd.read_excel("dados.xlsx",
sheet_name="Planilha1")

# JSON
df = pd.read_json("dados.json")

# URL direta!
url = "https://exemplo.com/dados.csv"
df = pd.read_csv(url)
```

Escrevendo DataFrames em Arquivos

```python
# Salvar como CSV
df.to_csv("resultado.csv",
          index=False)  # sem índice

# Excel
df.to_excel("resultado.xlsx",
            index=False)
```

Propriedades dos DataFrames

```python
df.head(5)
df.tail(3)
df.info()
df.describe()
df.shape
df.columns
df.isnull().sum()
```

Saídas

```
# primeiras 5 linhas
# últimas 3 linhas
# tipos + valores nulos
# estatísticas descritivas
# (linhas, colunas)
# lista de colunas
# contagem de NaN
```

Seleção e Filtro de Dados

```python
df.head(5)
df.tail(3)
df.info()
df.describe()
df.shape
df.columns
df.isnull().sum()
```

Saídas

```
# primeiras 5 linhas
# últimas 3 linhas
# tipos + valores nulos
# estatísticas descritivas
# (linhas, colunas)
# lista de colunas
# contagem de NaN
```

Busca de dados por índices e filtragem

```
# .loc - por RÓTULO
df.loc[0]            # linha 0
df.loc[0, "nome"]  # linha 0, col
nome
df.loc[0:2, "nota":"turma"]

# .iloc - por POSIÇÃO (0-based)
df.iloc[0]           # 1ª linha
df.iloc[0, 1]       # 1ª linha, 2ª col
df.iloc[:3, :2]     # 3 linhas, 2
colunas
```

Busca de dados por índices e filtragem

```python
# .query()
df.query("nota >= 8")
df.query("nota >= 7 and turma ==
'A'")

# Usando variável externa com @
nota_min = 8.0
df.query("nota >= @nota_min")
```

```python
# Filtro booleano
df[df["nota"] >= 8.0]

# Múltiplas condições
mask = (df["nota"] >= 7) & \
       (df["turma"] == "A")
df[mask]
```

Transformações de dados

```python
# Strings

df["nome"].str.upper()
df["nome"].str.lower()
df["nome"].str.strip()
df["nome"].str.contains("An")
df["nome"].str.replace(".", ",")
df["nome"].str.len()
df["nome"].str.split(" ")
```

```python
# Data/hora
# Converter para datetime
df["data"] =
pd.to_datetime(df["data"])

# Acessor .dt
df["data"].dt.year
df["data"].dt.month
df["data"].dt.day
df["data"].dt.day_name() #"Monday"...
df["data"].dt.quarter
```

Agrupamento e Agregação

```python
# Exemplo: dataset de vendas
vendas = pd.DataFrame({
    "vendedor":
["Ana","Ana","Bruno","Bruno"],
    "produto":  ["A","B","A","C"],
    "valor":    [100,200,150,300],
    "qtd":      [2,1,3,1]
})
```

```python
# Total por vendedor
vendas.groupby("vendedor")["valor"].sum()
# vendedor
# Ana      300
# Bruno    450

# Múltiplas agregações com agg
vendas.groupby("vendedor").agg(
    total    = ("valor", "sum"),
    media    = ("valor", "mean"),
    n_pedidos = ("valor", "count")
)
```

Agrupamento e Agregação

```
# Agrupar por múltiplas colunas
vendas.groupby(["vendedor","produto"]) \
      ["valor"].sum()

# value_counts - frequência de categorias
vendas["produto"].value_counts()
# A    2
# B    1
# C    1
```

```
# pivot_table - como tabela dinâmica do Excel
pd.pivot_table(
    vendas,
    values="valor",
    index="vendedor",
    columns="produto",
    aggfunc="sum",
    fill_value=0)
# Ordenar resultados
resultado = vendas.groupby("vendedor") \
                  ["valor"].sum() \
.sort_values(ascending=False)
```

“Limpeza” de dados

```python
# Checar valores nulos
df.isnull().sum()      # por coluna
df.isnull().sum().sum() # total

# Percentual de nulos
df.isnull().mean() * 100

# Remover linhas com qualquer NaN
df.dropna()
# Remover linhas só se NaN em
colunas específicas
df.dropna(subset=["nome",
"valor"])
```

```python
# Preencher NaN
df.fillna(0)                    # com zero
df["nota"].fillna(
    df["nota"].mean()       # com média
)
df.fillna(method="ffill")  # forward fill
df.fillna(method="bfill")  # bward fill
```

“Limpeza” de dados (2)

```python
# Detectar duplicatas
df.duplicated().sum()

# Remover duplicatas
df.drop_duplicates()
df.drop_duplicates(
    subset=["email"],
    keep="first"
)
```

```python
# Converter tipos
df["valor"] = df["valor"].astype(float)
df["qtd"]   = df["qtd"].astype(int)
df["data"]  = pd.to_datetime(df["data"])

# Substituir valores
df.replace({"sim": True, "nao": False})
```

Mais sobre Pandas

- Indexação avançada (IntervalIndex, PeriodIndex, set_index, …)
- Funções para séries temporais: resample, rolling, …
- “Limpeza” com expressões regulares
- Leitura de SQL, HTML, Google Sheets
- Saída gráfica nativa da Pandas (df.plot())

- Documentação: https://pandas.pydata.org/docs/

Projeto

Vamos trabalhar com uma base de dados real: os microdados do ENEM, disponibilizados pelo INEP aqui.

Recupere os microdados e localize o arquivo que contém os resultados individuais de todos os inscritos no exame. Identifique, através da documentação e do próprio arquivo, as colunas que trazem os seguintes dados:

1) Notas das provas específicas e notas da redação
2) UF, Cidade e Escola de origem do aluno
3) Tipo de Escola (privada, pública, …)

Faça uma carga parcial do arquivo, tratando apenas dessas colunas. Identifique dados inconsistentes ou faltantes. Produza dados agregados a partir dos dados brutos:

a) Média de notas por aluno
b) Média de notas por escola
c) Média de notas por município
d) Média de notas por UF

Produza um ranking dos 10 melhores alunos, 10 melhores escolas (geral e nos extratos públicos e privado) e 10 melhores municípios do país e do Rio Grande do Sul.